Cross-cluster Live Migration with Red Hat Advanced Cluster Management

2026/04/13

Table of Contents

1. Use-case description . . . 1
2. Preparations . . . 3
   2.1. Stretched L2 live-migration network scenario . . . 3
   2.2. Routed L3 live-migration networks scenario . . . 5
3. Implementation steps . . . 8
   3.1. Label clusters in RHACM . . . 8
   3.2. Enable `decentralizedLiveMigration` feature gate . . . 8
   3.3. Enabling Fine-Grained RBAC for RHACM . . . 9
   3.4. Enabling Migration Integration Components . . . 9
   3.5. Assigning the `kubevirt.io:admin` Role . . . 10
   3.6. Configuring the Live Migration Network . . . 10
   3.7. Implementation Example: Stretched L2 Network . . . 11
   3.8. Implementation Example: Routed L3 Networks . . . 12
   3.9. Assigning the Migration Network to HyperConverged (HCO) . . . 13
   3.10. Verifying the Migration Network Attachment . . . 14
4. Initiating Cross-Cluster Live Migration . . . 15
   4.1. Post-Migration Verification . . . 17
5. Troubleshooting . . . 19
   5.1. "Cross-cluster migration is not supported on this cluster" . . . 19
   5.2. Error: admission webhook "validate.mtv.plan" denied the request: User does not have permission to access the target namespace... . . . 20
   5.3. Error: Migration Stalled at 66% . . . 21
   5.4. Error: "authentication handshake failed: context deadline exceeded" . . . 21
   5.5. Verification Without Standard Networking Tools . . . 22
6. Appendix . . . 23

1. Use-case description

Red Hat Advanced Cluster Management (RHACM) is a management platform for Kubernetes and OpenShift. It handles the lifecycle of multiple clusters, enforces security policies, and provides visibility across environments.

As a central control plane, RHACM manages both Virtual Machines and the clusters hosting them, providing a single interface for workloads across the fleet. It also automates the discovery and configuration of clusters for MTV.

Migration Toolkit for Virtualization (MTV) is the engine that moves virtual machine workloads. While MTV is known for migrating VMs from external platforms like VMware or OpenStack to OpenShift, it also provides the capabilities required to perform easy live migrations between distinct OpenShift clusters.

Cross-cluster Live Migration (CCLM) is a workflow that moves running Virtual Machines (VMs) from one managed cluster to another with minimal to zero service interruption.

Objective	Demonstrate the ability of Red Hat Advanced Cluster Management (RHACM) to live-migrate Virtual Machines between distinct OpenShift clusters while ensuring state persistence and zero downtime.
Scope	This use-case covers the following activities: • Configuration of the clusters for CCLM. • Installation and configuration of the required CCLM components. • Basic troubleshooting of common migration hurdles.

Requirements	• **Clusters:** Red Hat OpenShift Container Platform (OCP) clusters (version 4.21 or later) installed and operational.
	• **Privileges:** `cluster-admin` access on all clusters taking part in the migration and the RHACM Hub.
	• **Networking:** A dedicated network for live migration must be configured to provide direct connectivity between the clusters taking part in the migration.
	• **RHACM:** is installed, configured and has the clusters taking part in the migration imported.
	• **Software Stack:**
	  ◦ Red Hat ACM (2.16+) installed on the Hub cluster.
	  ◦ Red Hat MTV (2.11+) installed on the Hub cluster.
	  ◦ Red Hat OpenShift Virtualization (4.21+) installed on all clusters taking part in the migration.

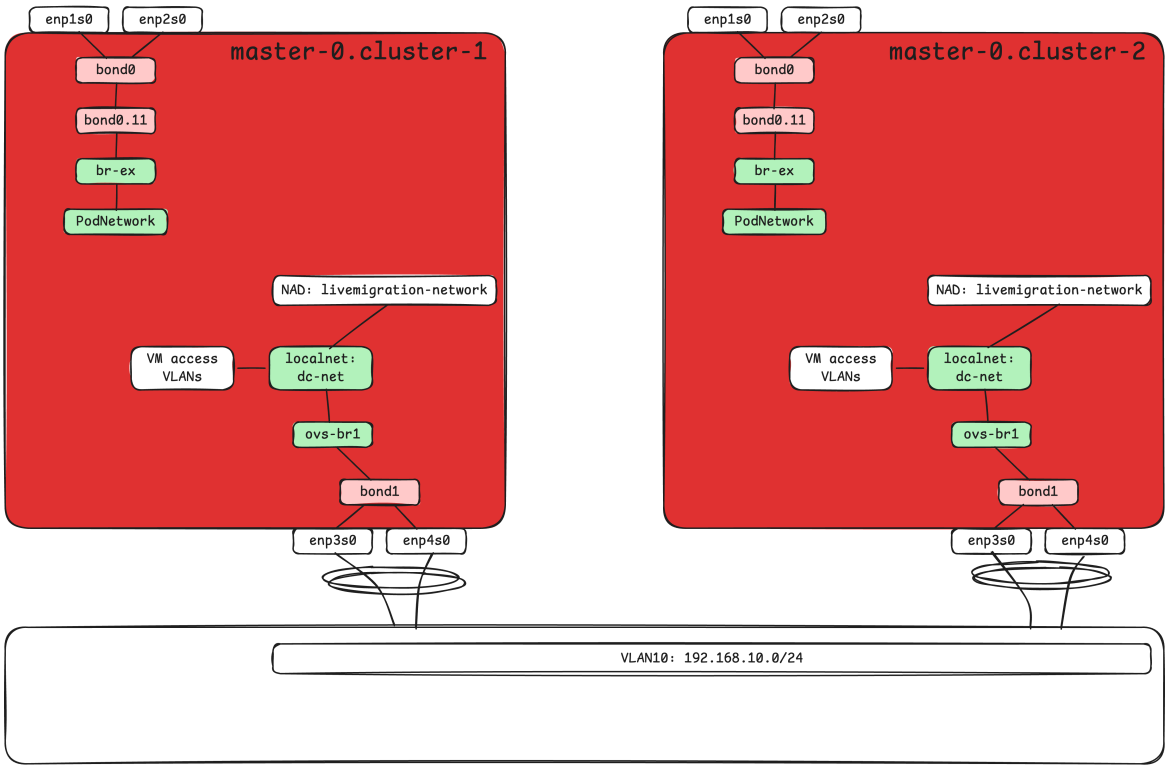
2. Preparations

To provide a clear reference for the networking setup, the following examples illustrate the network configurations used in this lab. Depending on your infrastructure, you will likely encounter one of two scenarios:

- **Scenario 1: Stretched L2 Network** In this setup, a single VLAN is stretched across both clusters. This is the simpler configuration, allowing for the use of either **OVN** or **Linux bridge** networks to handle migration traffic.
- **Scenario 2: Routed L3 Networks** In this setup, each cluster has its own separate, dedicated VLAN. Traffic between these VLANs is handled by a router.

IMPORTANT	For **Routed L3** scenario, you **MUST** use a **Linux bridge** based network. This is currently the only configuration that allows for the necessary gateway and routing specifications required to reach the remote cluster's migration subnet.

2.1. Stretched L2 live-migration network scenario



These examples demonstrate a simple scenario where bonded interfaces provide the foundation for OVN-only networking.

2.1.1. cluster-1 NNCP

```
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: master-0-bond1-vlan10
spec:
  desiredState:
    interfaces:
      - ipv4:
          dhcp: false
          enabled: false
        link-aggregation:
          mode: active-backup
          options:
            primary: enp3s0
          port:
            - enp3s0
            - enp4s0
        mtu: 9000
        name: bond1
        state: up
        type: bond
      - bridge:
          allow-extra-patch-ports: true
          options:
            stp: false
          port:
            - name: bond1
        name: ovs-br1
        state: up
        type: ovs-bridge
    ovn:
      bridge-mappings:
        - bridge: ovs-br1
          localnet: dc-net
          state: present
  nodeSelector:
    kubernetes.io/hostname: master-0.cluster-1.openshift.one
```

2.1.2. cluster-2 NNCP

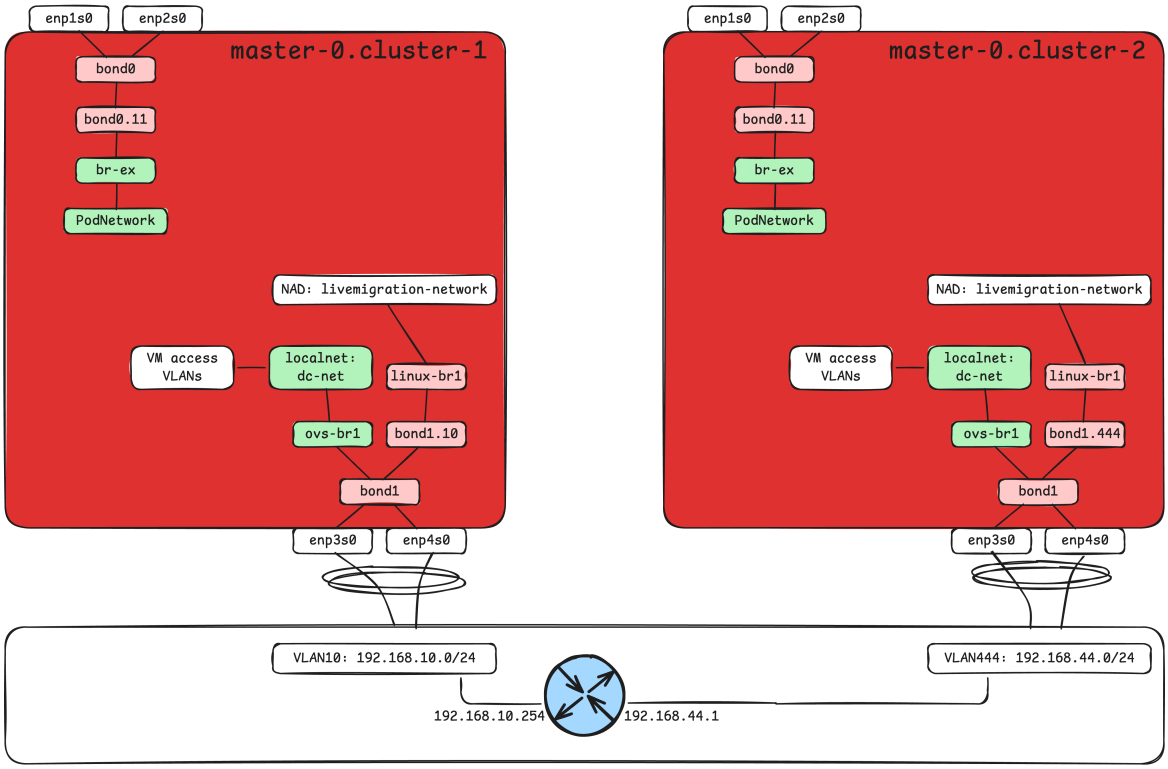
```
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: master-0-bond1-vlan10
spec:
  desiredState:
    interfaces:
      - ipv4:
          dhcp: false
          enabled: false
        link-aggregation:
          mode: active-backup
          options:
            primary: enp3s0
          port:
            - enp3s0
            - enp4s0
        mtu: 9000
        name: bond1
        state: up
        type: bond
      - bridge:
          allow-extra-patch-ports: true
          options:
            stp: false
          port:
```

```
            - name: bond1
        name: ovs-br1
        state: up
        type: ovs-bridge
    ovn:
      bridge-mappings:
        - bridge: ovs-br1
          localnet: dc-net
          state: present
  nodeSelector:
    kubernetes.io/hostname: master-0.cluster-2.openshift.one
```

IMPORTANT

In a stretched L2 configuration, you act as the IP Address Manager (IPAM). It is your responsibility to manually allocate and track IP addresses to avoid collisions within the shared live-migration VLAN. See more information in NetworkAttachmentDefinitions below.

2.2. Routed L3 live-migration networks scenario



These examples demonstrate a complex but common scenario where bonded interfaces provide the foundation for both OVN and Linux bridge networking.

2.2.1. cluster-1 NNCP

```
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: master-0-bond1-vlan10
spec:
```

```
  desiredState:
    interfaces:
      - name: bond1
        type: bond
        state: up
        mtu: 9000
        link-aggregation:
          mode: active-backup
          options:
            primary: enp3s0
          port:
            - enp3s0
            - enp4s0
      - name: ovs-br1
        type: ovs-bridge
        state: up
        bridge:
          allow-extra-patch-ports: true
          options:
            stp: false
          port:
            - name: bond1
      - name: bond1.10
        mtu: 8982
        type: vlan
        state: up
        vlan:
          base-iface: bond1
          id: 10
      - name: linux-br1
        type: linux-bridge
        state: up
        bridge:
          options:
            stp:
              enabled: false
          port:
            - name: bond1.10
    ovn:
      bridge-mappings:
        - bridge: ovs-br1
          localnet: dc-net
          state: present
  nodeSelector:
    kubernetes.io/hostname: master-0.cluster-1.openshift.one
```

2.2.2. cluster-2 NNCP

```
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: master-0-bond1-vlan444
spec:
  desiredState:
    interfaces:
      - name: bond1
        type: bond
        state: up
        mtu: 9000
        link-aggregation:
          mode: active-backup
          options:
            primary: enp3s0
          port:
            - enp3s0
            - enp4s0
      - name: ovs-br1
        type: ovs-bridge
        state: up
        bridge:
          allow-extra-patch-ports: true
          options:
```

```
            stp: false
          port:
            - name: bond1
      - name: bond1.444
        mtu: 8982
        type: vlan
        state: up
        vlan:
          base-iface: bond1
          id: 444
      - name: linux-br1
        type: linux-bridge
        state: up
        bridge:
          options:
            stp:
              enabled: false
          port:
            - name: bond1.444
    ovn:
      bridge-mappings:
        - bridge: ovs-br1
          localnet: dc-net
          state: present
  nodeSelector:
    kubernetes.io/hostname: master-0.cluster-2.openshift.one
```

NOTE

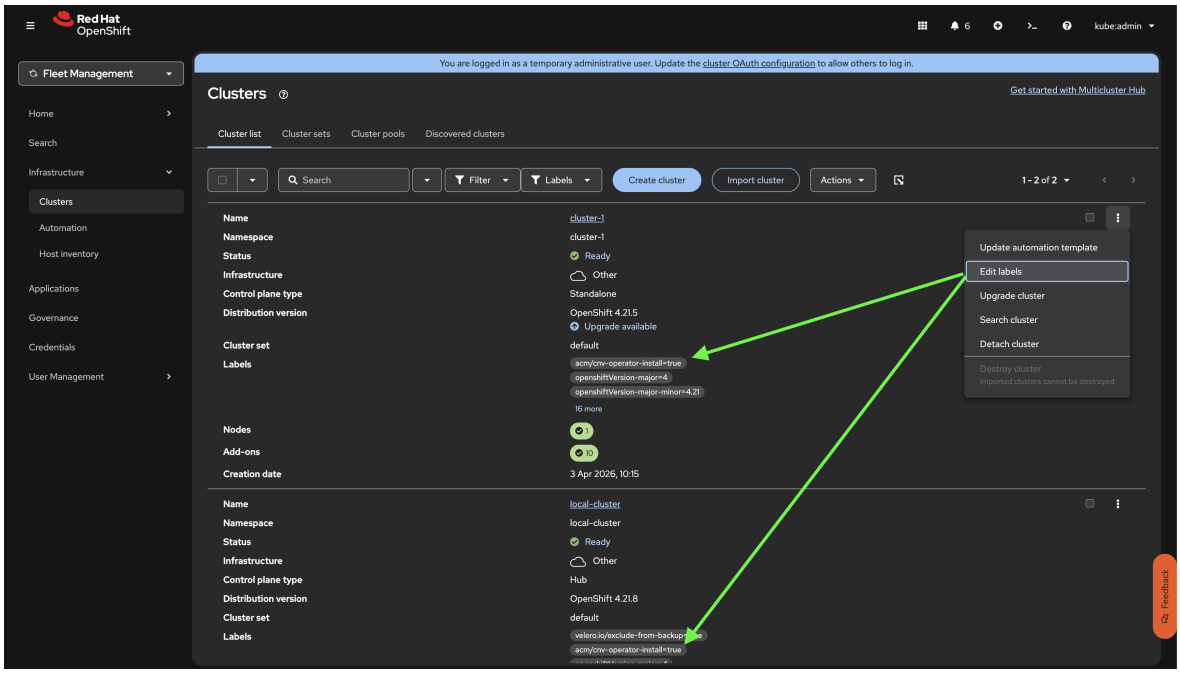
In this configuration:

- Interfaces `enp3s0` and `enp4s0` are aggregated into `bond1`.
- `ovs-br1` (OVS Bridge) is mapped to the bond to provide VM access to the data center network via OVN.
- A VLAN interface (`bond1.10` or `bond1.444`) is created on top of the bond to act as the transport for a Linux bridge (`linux-br1`).
- The **`linux-br1`** bridge is used specifically for the dedicated live migration network.

3. Implementation steps

3.1. Label clusters in RHACM

To enable the necessary environment, navigate to the **Fleet Management** of RHACM and then go to **Infrastructure** > **Clusters**. For each of the clusters you want to live-migrate VMs between, use the **Edit labels** action to add the `acm/cnv-operator-install=true` label. The label acts as a policy trigger that tells ACM to automatically deploy and configure the OpenShift Virtualization (CNV) Operator on a managed cluster.



3.2. Enable `decentralizedLiveMigration` feature gate

To support CCLM, you must enable the `decentralizedLiveMigration` feature gate. Ensure the value is set to `true` within the `HCO` (HyperConverged Cluster Operator) configuration on both participating clusters:

```
spec:
  featureGates:
    decentralizedLiveMigration: true
```

You can simply patch HCO on both clusters running the following command:

```
oc -n openshift-cnv  patch hco kubevirt-hyperconverged --type='merge' -p
'{"spec":{"featureGates":{"decentralizedLiveMigration":true}}}'
```

3.3. Enabling Fine-Grained RBAC for RHACM

Cross-cluster Live Migration requires fine-grained role-based access control (RBAC) to be enabled. Run it only on the cluster where RHACM HUB is running.

1. Identify the index of the `fine-grained-rbac` component within the `MultiClusterHub` (MCH) resource and enable it:

```
# Identify the component index
INDEX=$(oc get mch multiclusterhub -n open-cluster-management -o json | jq '.spec.overrides.components |
map(.name == "fine-grained-rbac") | index(true)')

# Enable the component using the identified index
oc patch mch multiclusterhub -n open-cluster-management --type=json \
  -p="[{\"op\": \"replace\", \"path\": \"/spec/overrides/components/$INDEX/enabled\", \"value\": true}]"
```

NOTE For more details on fine-grained RBAC configuration, refer to the RHACM Documentation.

3.4. Enabling Migration Integration Components

To enable the integration between Red Hat Advanced Cluster Management (RHACM) and Migration Toolkit for Virtualization (MTV), you must enable the `cnv-mtv-integrations` component.

1. Identify the index for the `cnv-mtv-integrations` component and update the `MultiClusterHub` (MCH) configuration. Run it only on the cluster where RHACM HUB is running.

```
# Identify the component index
INDEX=$(oc get mch multiclusterhub -n open-cluster-management -o json | jq '.spec.overrides.components |
map(.name == "cnv-mtv-integrations") | index(true)')

# Enable the component
oc patch mch multiclusterhub -n open-cluster-management --type=json \
  -p="[{\"op\": \"replace\", \"path\": \"/spec/overrides/components/$INDEX/enabled\", \"value\": true}]"
```

NOTE For more information on VM migration prerequisites, see the RHACM Virtualization documentation.

3.5. Assigning the `kubevirt.io:admin` Role

A critical prerequisite for CCLM is ensuring that the user identity RHACM uses to manage the managed clusters has the `kubevirt.io:admin` role assigned.

IMPORTANT	Even if the user (e.g., `kubeadmin`) is already a member of the `system:cluster-admins` group, you **must** explicitly bind the `kubevirt.io:admin` role. Failure to do so results in the following error from the `validate.mtv.plan` admission webhook: `User does not have permission to access the target namespace`

To automate this across your fleet, create a `ClusterPermission` object for each cluster involved in the migration:

```
apiVersion: rbac.open-cluster-management.io/v1alpha1
kind: ClusterPermission
metadata:
  name: kubevirt-admin
  namespace: <cluster_name> ①
spec:
  clusterRoleBinding:
    roleRef:
      apiGroup: rbac.authorization.k8s.io
      kind: ClusterRole
      name: kubevirt.io:admin
    subject:
      kind: Group
      name: system:cluster-admins ②
      apiGroup: rbac.authorization.k8s.io
```

① Replace `<cluster_name>` with the name of the namespace where managed cluster is defined in RHACM.

② If you are using a different user or identity provider, adjust the `subject` to match your environment.

3.6. Configuring the Live Migration Network

To perform Cross-Cluster Live Migration (CCLM), participating clusters must have a dedicated migration network that allows direct communication between virtualization components, specifically the `virt-handler` and `virt-synchronization-controller` pods.

NOTE	Since `virt-handler` runs on every node capable of hosting VMs, you must ensure your IP address layout accounts for the total number of nodes across the fleet.

Using a dedicated network is recommended to prevent migration traffic from impacting production workloads. Depending on your infrastructure, choose one of the following two approaches:

3.6.1. Stretched L2 Network

A single L2 network stretched across both clusters (e.g., via OVN-based User Defined Networks (UDN) or a physical Linux bridge).

- **Requirement:** Ensure each cluster uses a unique, non-overlapping IP address range within the same subnet to avoid collisions.
- **Configuration:** Set the NetworkAttachmentDefinitions' annotation `forklift.konveyor.io/route: none`.

3.6.2. Routed L3 Networks

Separate L2 networks for each cluster, connected via a gateway.

- **Requirement:** Must use a Linux bridge-based network (as OVN-K is limited regarding custom gateway specifications in this context).
- **Configuration:** Set the NetworkAttachmentDefinitions' annotation `forklift.konveyor.io/route: <gateway_ip>`.

3.7. Implementation Example: Stretched L2 Network

The following example demonstrates how to configure the `NetworkAttachmentDefinition` (NAD) for two clusters using a stretched network.

3.7.1. cluster-1 Configuration

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: livemigration-network-cluster-1
  namespace: openshift-cnv
  annotations:
    forklift.konveyor.io/route: none
spec:
  config: |-
    {
        "cniVersion": "0.4.0",
        "name": "livemigration-network",
        "type": "ovn-k8s-cni-overlay",
        "physicalNetworkName": "dc-net",
        "mtu": 8982,
        "netAttachDefName": "openshift-cnv/livemigration-network",
        "topology": "localnet",
        "vlanID": 10,
        "subnets": "192.168.10.0/24",
        "excludeSubnets": "192.168.10.0/27, 192.168.10.64/27, 192.168.10.96/27, 192.168.10.128/27,
192.168.10.160/27, 192.168.10.192/27, 192.168.10.224/27" ①
    }
```

① Only 192.168.10.32/27 can be used in this NAD to avoid IP address collision with cluster-2.

3.7.2. cluster-2 Configuration

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: livemigration-network-cluster-2
  namespace: openshift-cnv
  annotations:
    forklift.konveyor.io/route: none
spec:
  config: |-
    {
        "cniVersion": "0.4.0",
        "name": "livemigration-network",
        "type": "ovn-k8s-cni-overlay",
        "physicalNetworkName": "dc-net",
        "mtu": 8982,
        "netAttachDefName": "openshift-cnv/livemigration-network",
        "topology": "localnet",
        "vlanID": 10,
        "subnets": "192.168.10.0/24",
        "excludeSubnets": "192.168.10.0/27, 192.168.10.32/27, 192.168.10.96/27, 192.168.10.128/27,
192.168.10.160/27, 192.168.10.192/27, 192.168.10.224/27" ①
    }
```

① Only 192.168.10.64/27 can be used in this NAD to avoid IP address collision with cluster-1.

3.8. Implementation Example: Routed L3 Networks

The following example demonstrates how to configure the `NetworkAttachmentDefinition` (NAD) for two clusters using a routed approach.

3.8.1. cluster-1 Configuration

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: livemigration-network-cluster-1
  namespace: openshift-cnv
  annotations:
    forklift.konveyor.io/route: 192.168.10.254 ①
spec:
  config: |
    {
      "cniVersion": "0.3.1",
      "name": "livemigration-network-cluster-1",
      "type": "bridge",
      "bridge": "linux-br1",
      "mtu": 8982,
      "isGateway": true,
      "ipam": {
        "type": "whereabouts",
        "range": "192.168.10.0/24",
        "exclude": [
          "192.168.10.0/27", "192.168.10.32/27", "192.168.10.64/27",
          "192.168.10.128/27", "192.168.10.160/27", "192.168.10.192/27",
          "192.168.10.224/27"
```

```
        ],
        "routes": [
          {
            "dst": "192.168.44.0/24", ②
            "gw": "192.168.10.254" ①
          }
        ]
      }
    }
```

① The local gateway IP for Cluster 1.

② The destination subnet for the Cluster 2 migration network.

3.8.2. cluster-2 Configuration

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: livemigration-network-cluster-2
  namespace: openshift-cnv
  annotations:
    forklift.konveyor.io/route: 192.168.44.1 ①
spec:
  config: |
    {
      "cniVersion": "0.3.1",
      "name": "livemigration-network-cluster-2",
      "type": "bridge",
      "bridge": "linux-br1",
      "mtu": 8982,
      "isGateway": true,
      "ipam": {
        "type": "whereabouts",
        "range": "192.168.44.0/24",
        "exclude": [
          "192.168.44.0/27", "192.168.44.32/27", "192.168.44.64/27",
          "192.168.44.128/27", "192.168.44.160/27", "192.168.44.192/27",
          "192.168.44.224/27"
        ],
        "routes": [
          {
            "dst": "192.168.10.0/24", ②
            "gw": "192.168.44.1" ①
          }
        ]
      }
    }
```

① The local gateway IP for Cluster 2.

② The destination subnet for the Cluster 1 migration network.

3.9. Assigning the Migration Network to HyperConverged (HCO)

Once the `NetworkAttachmentDefinition` (NAD) is created, you must configure the `HyperConverged` (HCO) resource on each cluster to use the respective network for live migration.

1. On **Cluster 1**, patch the HCO to use its local migration network:

```
oc patch hyperconverged kubevirt-hyperconverged -n openshift-cnv --type merge \
  -p '{ "spec": { "liveMigrationConfig": { "network": "livemigration-network-cluster-1" }}}'
```

2. On **Cluster 2**, apply the corresponding patch:

```
oc patch hyperconverged kubevirt-hyperconverged -n openshift-cnv --type merge \
  -p '{ "spec": { "liveMigrationConfig": { "network": "livemigration-network-cluster-2" }}}'
```

IMPORTANT

Applying this change triggers an automatic restart of the `virt-handler` and `virt-synchronization-controller` pods. This is necessary to attach the new network interface to the infrastructure pods.

3.10. Verifying the Migration Network Attachment

After the pods have restarted, verify that the `migration0` interface has been successfully assigned and has received an IP address from the defined range.

Check the `virt-synchronization-controller` network status:

```
oc -n openshift-cnv get pod -l kubevirt.io=virt-synchronization-controller -o json | \
jq '.items[] | .metadata.annotations["k8s.v1.cni.cncf.io/network-status"] | fromjson | .[] | select(.interface ==
"migration0")'
```

Check the `virt-handler` network status:

```
oc -n openshift-cnv get pod -l kubevirt.io=virt-handler -o json | \
jq '.items[] | .metadata.annotations["k8s.v1.cni.cncf.io/network-status"] | fromjson | .[] | select(.interface ==
"migration0")'
```

Expected output should show the `migration0` interface linked to your specific NAD and an IP address within your configured subnet (lack of the routing information is expected):

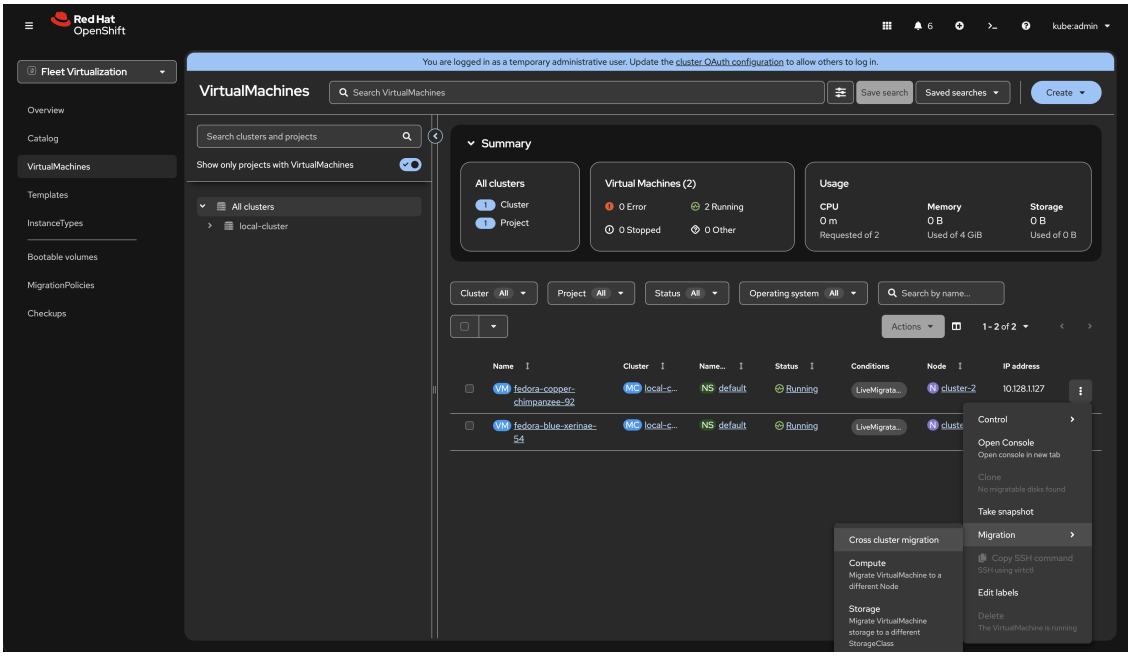
```
{
  "name": "openshift-cnv/livemigration-network-cluster-2",
  "interface": "migration0",
  "ips": [
    "192.168.44.96"
  ],
  "mac": "2a:a2:26:15:a1:9a",
  "dns": {}
}
```

With the networking confirmed and the RBAC in place, your environment is now ready to initiate Cross-Cluster Live Migrations.

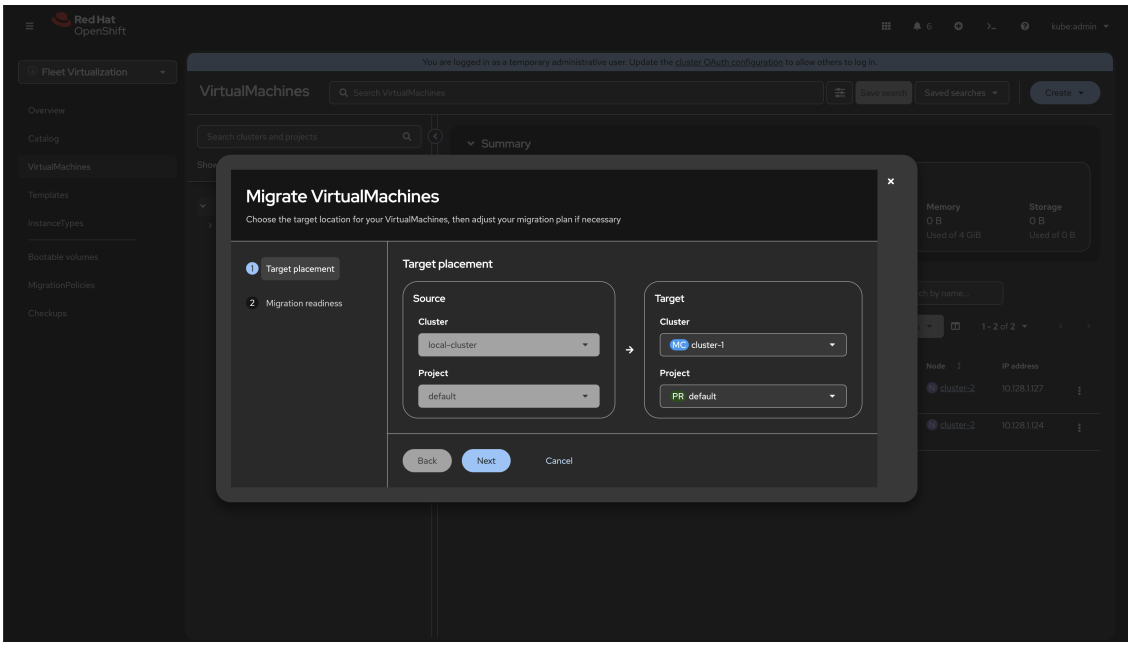
4. Initiating Cross-Cluster Live Migration

Follow these steps to trigger a migration from the RHACM Hub console.

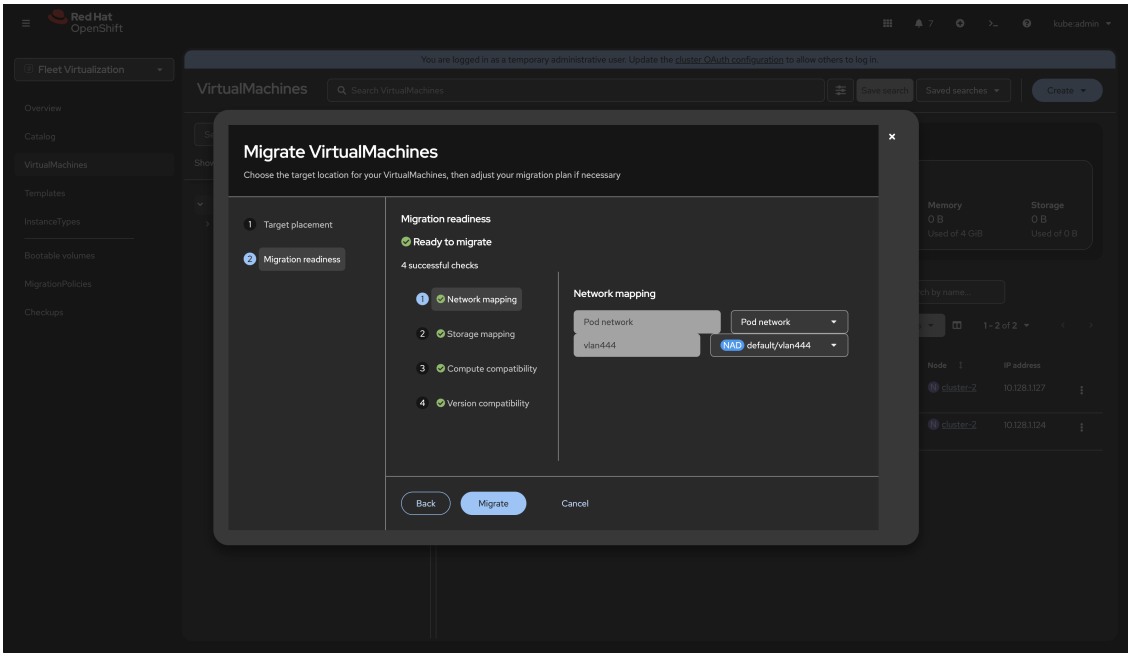
1. **Navigate to the Virtual Machines View:** From the RHACM Hub cluster console, go to **Fleet Virtualization** > **Virtual Machines**.
2. **Trigger the Migration:** Locate the VM you wish to migrate, click the **three-dots menu** (vertical ellipsis) on the right-hand side, and select **Migrate** > **Cross-cluster migration**.



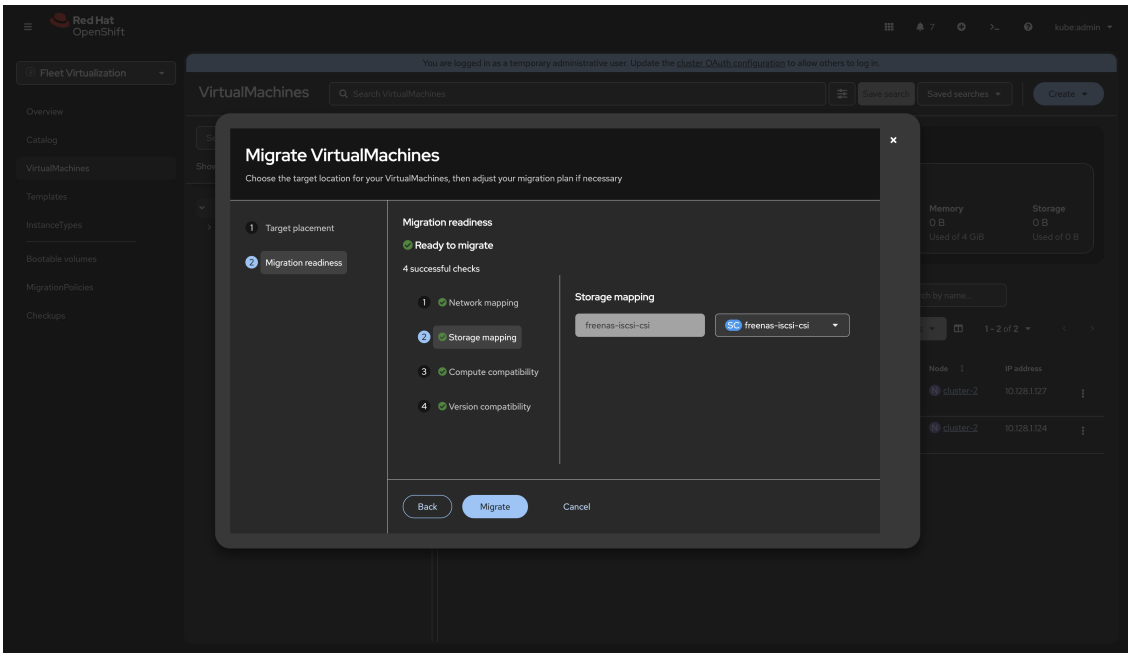
3. **Define the Destination:** Specify the target cluster and the destination project where the VM will reside.



4. **Configure Network Mappings:** Review or modify the network mappings. A network map defines how the virtual networks from the source cluster translate to the available networks on the destination cluster.

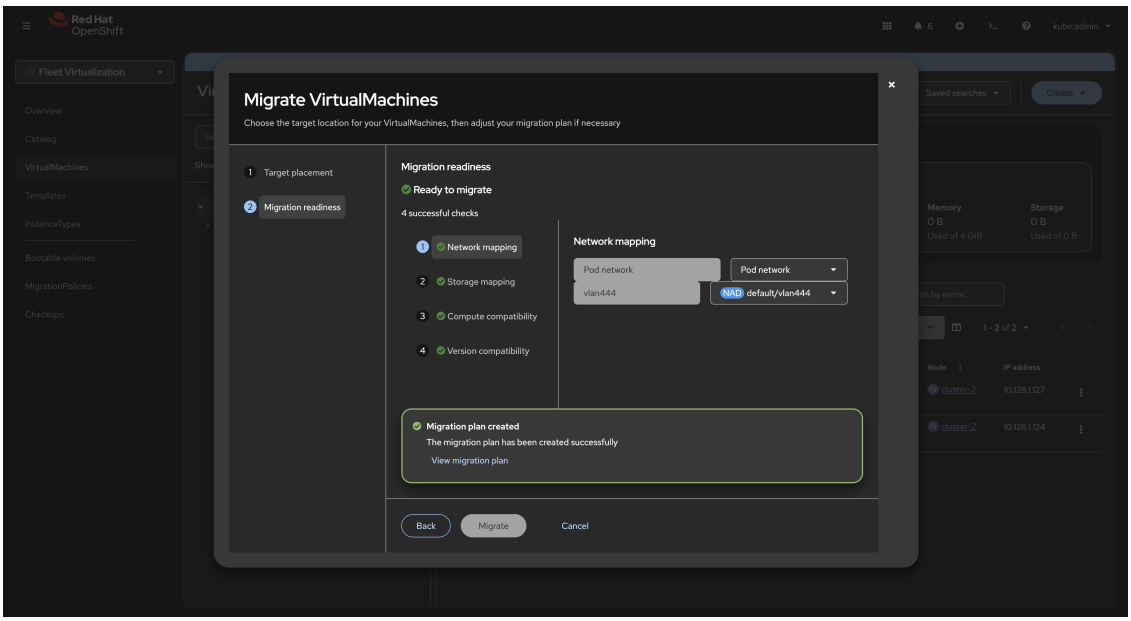


5. **Configure Storage Mappings:** Review or modify the storage mappings. A storage map defines how the storage resources (Persistent Volume Claims) on the source cluster are mapped to the available storage classes on the destination cluster.

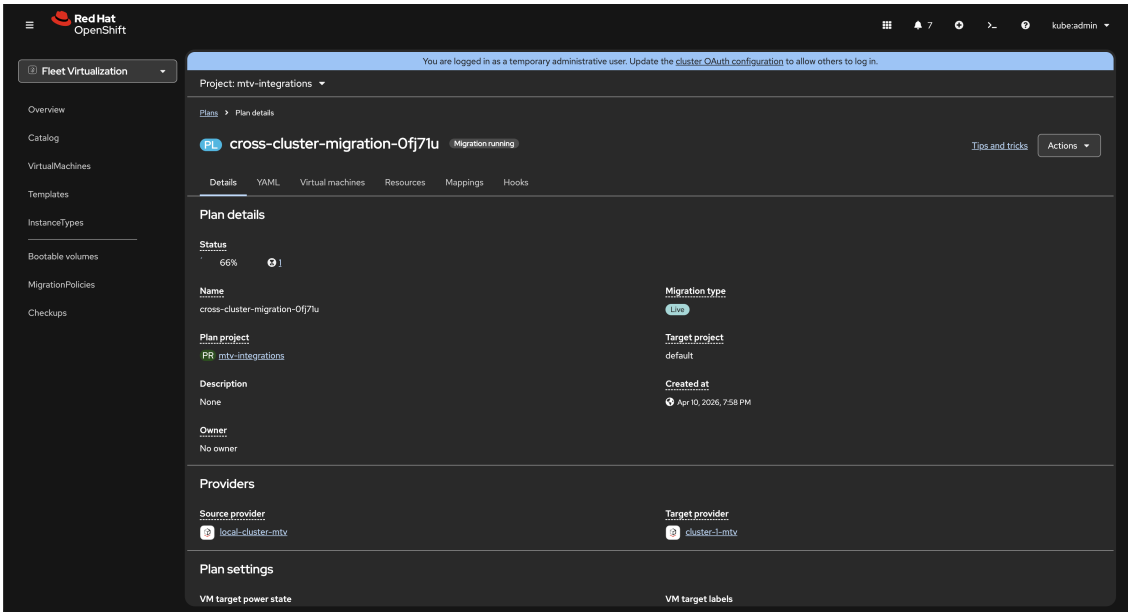


6. **Compatibility Check:** Verify the **Compute** and **Version** compatibility results. If the status is green, the destination cluster supports the VM's hardware requirements and software versions.

7. **Create and Run the Plan:** Select **Create** to generate the migration plan. You can then click **View migration plan** to monitor the status.



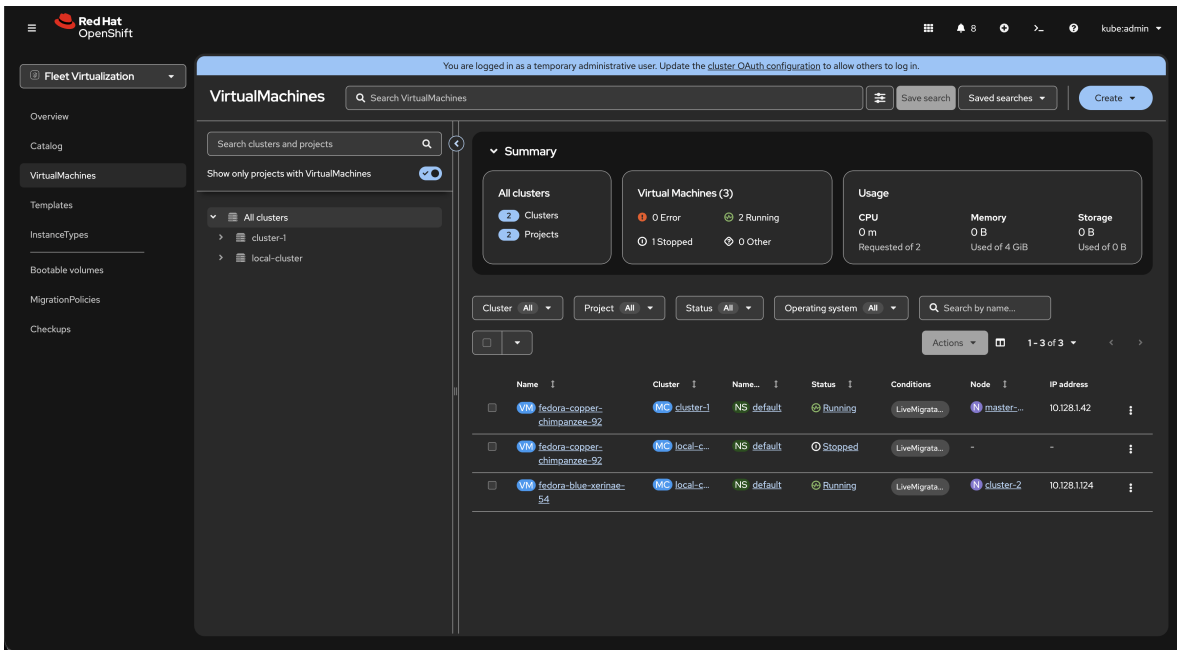
8. **Monitor Progress:** Observe the migration as it proceeds through the pre-copy and cutover phases.



4.1. Post-Migration Verification

Once the process is complete, the VM on the source cluster remains in a **Stopped** state (reflecting its status at the time of the cutover).

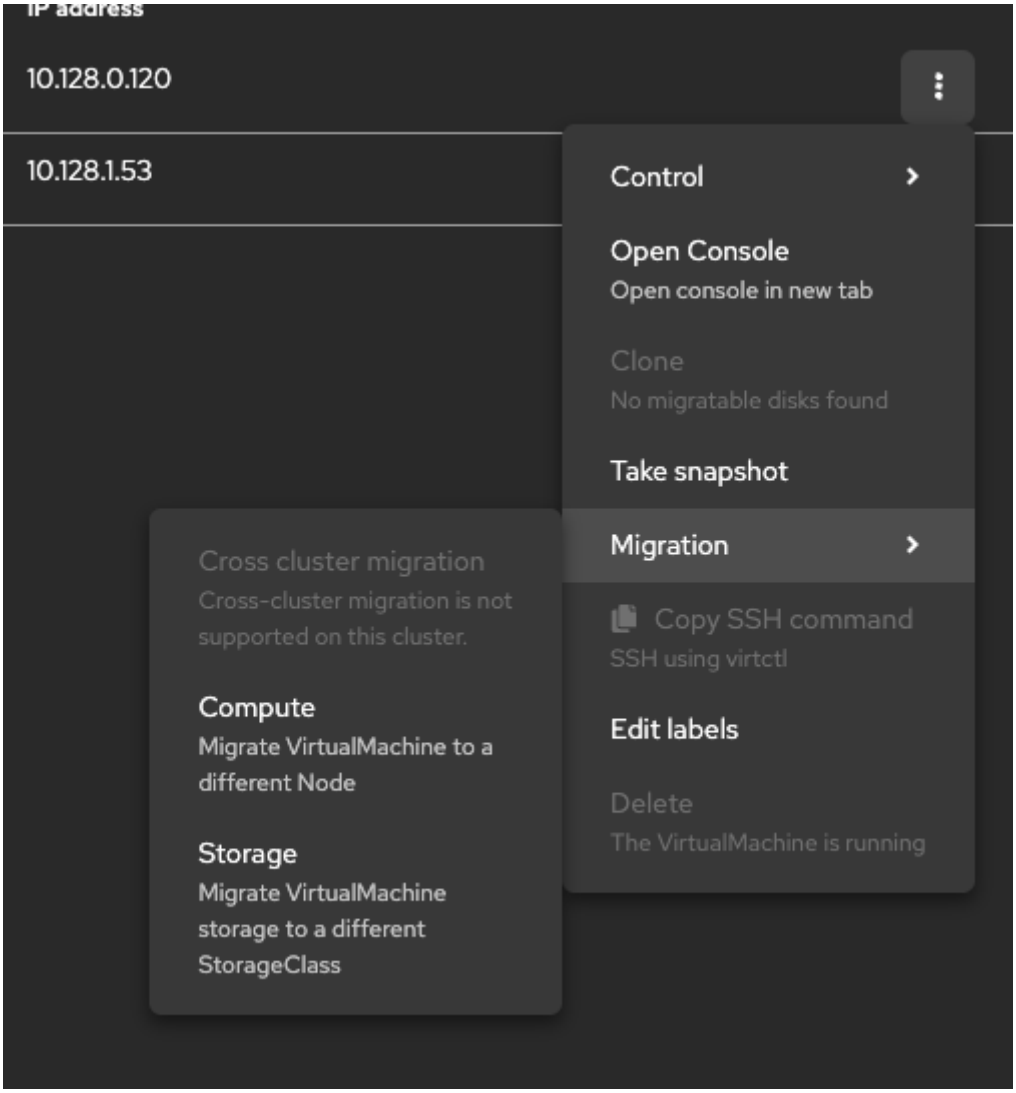
1. Verify that the VM is running correctly on the destination cluster.
2. After confirming that the workload is healthy, you may manually delete the VM resource from the source cluster to free up resources.


Red Hat OpenShift
kube:admin
Fleet Virtualization
Overview
Catalog
VirtualMachines
Templates
InstanceTypes
Bootable volumes
MigrationPolicies
Checkups
You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.
VirtualMachines
Search VirtualMachines
Save search
Saved searches
Create
Search clusters and projects
Show only projects with VirtualMachines
All clusters
cluster-1
local-cluster
Summary
All clusters
2 Clusters
2 Projects
Virtual Machines (3)
0 Error
2 Running
1 Stopped
0 Other
Usage
CPU
0 m
Requested of 2
Memory
0 B
Used of 4 GiB
Storage
0 B
Used of 0 B
Cluster All
Project All
Status All
Operating system All
Search by name...
Actions
1 - 3 of 3
Name
Cluster
Name...
Status
Conditions
Node
IP address
fedora-copper-chimpanzee-92
cluster-1
default
Running
LiveMigrata...
master-...
10.128.1.42
fedora-copper-chimpanzee-92
local-c...
default
Stopped
LiveMigrata...
fedora-blue-xerinae-54
local-c...
default
Running
LiveMigrata...
cluster-2
10.128.1.124

5. Troubleshooting

5.1. "Cross-cluster migration is not supported on this cluster"

When attempting to initiate a migration, the RHACM console may display a warning stating that cross-cluster migration is not supported for the selected cluster.



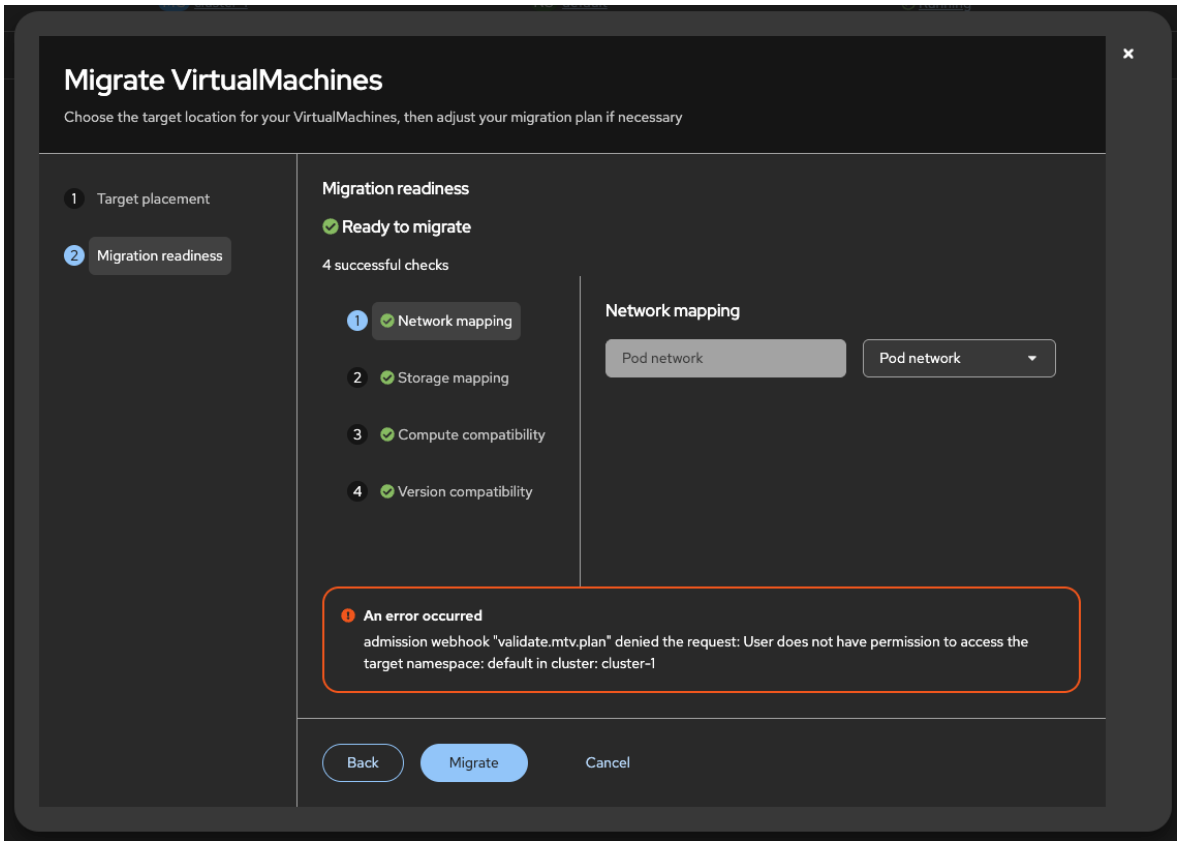
To resolve this issue, verify the following configuration requirements:

- **Component Integration:** Ensure the `cnv-mtv-integrations` component is enabled within the `MultiClusterHub` configuration.
- **RBAC Configuration:** Confirm that **fine-grained role-based access control** for virtualization is enabled in RHACM.
- **Feature Gates:** Verify that the `decentralizedLiveMigration` feature gate is set to `true` in the `HyperConverged` (HCO) resource on **both** the source and destination clusters.

- **Labeling:** Ensure the managed clusters are correctly labeled with `acm/cnv-operator-install=true` to allow RHACM to recognize them as migration targets.

5.2. Error: admission webhook "validate.mtv.plan" denied the request: User does not have permission to access the target namespace...

This error typically occurs during the final stage of migration plan creation when the MTV controller attempts to validate access to the target environment.



Symptom:

> An error occurred: admission webhook "validate.mtv.plan" denied the request: User does not have permission to access the target namespace: <namespace> in cluster: <cluster_name>

Resolution: This is a manifestation of missing RBAC permissions, even if the user has `cluster-admin` rights. To resolve this:

1. **Verify the Role Binding:** Ensure the `kubevirt.io:admin` role is explicitly assigned to the user or group RHACM uses to manage the target cluster.
2. **Check the ClusterPermission Object:** Confirm that a `ClusterPermission` resource (mentioned earlier in this document) exists on the Hub cluster in the managed

cluster's namespace.

5.3. Error: Migration Stalled at 66%

A migration that stalls at 66% typically indicates that pods on the source cluster which are taking part in the migration process are unable to reach pods on the destination cluster.

Symptom: The migration progress reaches 66% and stops. This occurs when the `virt-synchronization-controller` cannot establish a connection to the remote peer, often because it is attempting to use the incorrect network or because traffic is blocked.

Diagnostic: Check the logs of the `virt-synchronization-controller` on both clusters. Look for errors where the controller is "dialing" an IP address and receiving a connection refused or timeout error.

```
oc -n openshift-cnv logs -f deployment/virt-synchronization-controller
```

Example log output showing a connection failure: In this example, the controller is attempting to reach an IP (`10.128.1.59`) that is not accessible from the source cluster:

```
{"level":"info","msg":"creating new synchronization grpc
connection","outbound":"10.128.1.59:9185","timestamp":"2026-04-03T14:37:04.713771Z"}
{"level":"info","msg":"reenqueuing VMI...","reason":"rpc error: code = Unavailable desc = connection error: dial
tcp 10.128.1.59:9185: connect: connection refused"}
```

Resolution: If the logs show the controller is using the wrong IP range (e.g., the Pod network instead of the Migration network), or if you have recently updated your `NetworkAttachmentDefinition`, you must restart the virtualization components to apply the changes:

```
oc -n openshift-cnv rollout restart daemonset/virt-handler deployment/virt-synchronization-controller
```

NOTE If the logs show the **correct** Migration Network IP but still report "connection refused," verify that your physical network infrastructure or firewalls are not blocking the required ports between clusters.

5.4. Error: "authentication handshake failed: context deadline exceeded"

If the migration stalls and the logs show repeated authentication handshake failures, it often points to an underlying Maximum Transmission Unit (MTU) issue rather than an actual credential problem.

Symptom: The `virt-synchronization-controller` logs show the following rpc error:

> "transport: authentication handshake failed: context deadline exceeded"

Diagnostic: This error typically occurs when the TLS handshake packets - which are larger than standard data packets - exceed the allowed MTU of the network path between clusters. If a switch or router along the path cannot handle the packet size and "Fragmentation Needed" messages are ignored, the connection will time out.

Example logs:

```
{"level":"info","msg":"reenqueuing VMI...","reason":"rpc error: code = Unavailable desc = connection error: desc =
\"transport: authentication handshake failed: context deadline exceeded\"","timestamp":"2026-04-
13T14:51:15.323597Z"}
```

Resolution: * **Verify MTU consistency:** Ensure that the MTU settings in your `NodeNetworkConfigurationPolicy` (e.g., `mtu: 9000` for Jumbo Frames) match across all physical switches and routers between the clusters.

5.5. Verification Without Standard Networking Tools

In many cases pods are "minimal" images that do not contain troubleshooting utilities like `ping`, `curl`, `telnet`, or `nc`.

If you need to verify that a pod can reach the remote cluster's migration IP and port, you can use the built-in `/dev/tcp` device file to attempt a connection.

Resolution: Execute the following command from within the pod (e.g., using `oc exec` or `oc debug`):

```
# Replace with your destination IP and migration port (e.g., 9185)
timeout 5s bash -c 'echo > /dev/tcp/192.168.10.10/9185' && echo "Connection successful" || echo "Connection failed"
```

NOTE	If the command returns "Connection successful," the network path is open and the remote service is listening. If it hangs or returns "Connection failed," check your `NetworkAttachmentDefinition`, routing tables, or physical firewalls.

6. Appendix